

ZeroPad2d#

<https://pytorch.org/docs/stable/generated/torch.nn.ZeroPad2d.html#torch.nn.Z>

Description:

Pads the input tensor boundaries with zero. For N-dimensional padding, use `torch.nn.functional.pad()`. `padding` (int, tuple) – the size of the padding. If is int, uses the same padding in all boundaries. If a 4-tuple, uses `(padding_left\text{padding\_left}padding_left, padding_right\text{padding\_right}padding_right, padding_top\text{padding\_top}padding_top, padding_bottom\text{padding\_bottom}padding_bottom)` Input: (N, C, H_{in}, W_{in}) or (C, H_{in}, W_{in}) . Output: (N, C, H_{out}, W_{out}) or (C, H_{out}, W_{out}) , where

Function Signature

```
class torch.nn.ZeroPad2d(padding)[source]#
```

Parameters

Shape:

```
extra_repr()[source]#
```

Return type

Returns:

str

Code Examples

```

>>> m = nn.ZeroPad2d(2)
>>> input = torch.randn(1, 1, 3, 3)
>>> input
tensor([[[[-0.1678, -0.4418,  1.9466],
          [ 0.9604, -0.4219, -0.5241],
          [-0.9162, -0.5436, -0.6446]]]])
>>> m(input)
tensor([[[[ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,
            0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,
            0.0000],
          [ 0.0000,  0.0000, -0.1678, -0.4418,  1.9466,  0.0000,
            0.0000],
          [ 0.0000,  0.0000,  0.9604, -0.4219, -0.5241,  0.0000,
            0.0000],
          [ 0.0000,  0.0000, -0.9162, -0.5436, -0.6446,  0.0000,
            0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,
            0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,
            0.0000]]]])
>>> # using different paddings for different sides
>>> m = nn.ZeroPad2d((1, 1, 2, 0))
>>> m(input)
tensor([[[[ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
          [ 0.0000, -0.1678, -0.4418,  1.9466,  0.0000],
          [ 0.0000,  0.9604, -0.4219, -0.5241,  0.0000],
          [ 0.0000, -0.9162, -0.5436, -0.6446,  0.0000]]]])

```

Detailed Documentation

Documentation

Pads the input tensor boundaries with zero.

For N-dimensional padding, use `torch.nn.functional.pad()`.

`padding` (int, tuple) – the size of the padding. If is int, uses the same padding in all boundaries. If a 4-tuple, uses `(padding_left\text{padding\_left}padding_left, padding_right\text{padding\_right}padding_right, padding_top\text{padding\_top}padding_top, padding_bottom\text{padding\_bottom}padding_bottom)`

Input: (N,C,H_{in},W_{in}) (N, C, H_{in}, W_{in}) or (C,H_{in},W_{in}) (C, H_{in}, W_{in}) (C,H_{in},W_{in}) .

Output: (N,C,H_{out},W_{out}) (N, C, H_{out}, W_{out}) or (C,H_{out},W_{out}) (C, H_{out}, W_{out}) (C,H_{out},W_{out}) , where

$H_{out}=H_{in}+padding_top+padding_bottom$ $H_{out} = H_{in} + \text{padding_top} + \text{padding_bottom}$

$W_{out}=W_{in}+padding_left+padding_right$ $W_{out} = W_{in} + \text{padding_left} + \text{padding_right}$

Return the extra representation of the module.